

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum^{ORCID}^{1,2*}, Bart Buelens^{ORCID}¹, Ruben Taelman^{ORCID}², Gökhan Ertaylan^{ORCID}¹

²IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.

¹Digital Biological Systems, Flemish institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

*Corresponding author(s). E-mail(s): elias.crum@ugent.be;

Abstract

Purpose: The Variant Call Format (VCF) is the dominant standard for genomic variation, but a VCF file has no identifiers it shares with other files or with external knowledge, and no place for the rules that govern who may see its contents. The Resource Description Framework (RDF) provides both. We ask what converting VCF to RDF costs, whether the conversion is faithful, and what it lets a user do that a conventional pipeline does not.

Methods: VCF-RDFizer, distributed through PyPI, Conda-Forge and Docker, converts VCF into RDF in the public VCF Core vocabulary, optionally builds HDT and COTTAS representations, and ships plug-ins that link variants to shared identifiers and attach ODRL consent policies. Its validation layer compares thirteen SPARQL answers with a VCF parser’s and applies SHACL shapes. We measured conversion and retrieval on a ten-file public corpus, record and sample ladders and a whole genome, and asked one clinical question of real genomes by an RDF route and by a bcftools route.

Results: Asked which participants carry a ClinVar-classified variant in the 81 ACMG secondary-findings genes, and what each of three requesters may see under simulated consents, the RDF route joined genomes to ClinVar through SPDI identifiers and returned exactly the bcftools route’s carriers for every requester, over five heterogeneous genomes, 104 1000 Genomes participants, and a whole genome of 668 million triples. Its rules took 72 lines against 131, and a rule on one variant needed no code. On fixtures and slices of up to 17.1M triples, 984 of 988 paired comparisons were exactly equal and 4 were not applicable; the queries detected 96 of 113 injected corruptions and the full set of shapes all 113, though the default shape profile, the only one cheap enough for large inputs,

added none. The costs are real. A whole genome took 16.07 h to convert, 14.5 h of it building HDT and COTTAS, which the default query engine uses only as transport formats, and its outputs were 78× the downloaded file. Once its index was built, SPARQL answered single questions 2.5–2300× faster than an 11.5 s parser scan; but the parser wins any single cold question, converting a 100,000-record slice breaks even only after about 44 questions, and tabix wins narrow regional lookups.

Conclusion: Converting VCF to RDF does not pay in speed or in lines of code. It pays where variant identity, external meaning and consent must be explicit, shared across files and checkable against the source, and VCF-RDFizer makes that conversion reproducible and verifiable.

Keywords: Knowledge Representation, Genomic Variant Data, Variant Call Format, RDF, SPARQL, ODRL, HDT, COTTAS, Semantic Validation

1 Introduction

Variant Call Format. The Variant Call Format (VCF) is the dominant standard for representing and exchanging genomic variants across research and clinical sequencing, including pharmacogenomics and precision medicine [1–3]. Variant data are increasingly interpreted alongside external biomedical resources, sample metadata and governance constraints, and operational semantic-genomics initiatives already link variant observations to curated databases and controlled-access frameworks [4, 5]. Three of these needs strain the format. Clinical interpretation links variants to external knowledge, but VCF offers annotation fields rather than machine-interpretable links. Genomic data often need access control by region or by variant, which VCF does not support. And a question across several VCF files is usually answered with command-line utilities such as bcftools [6] or a custom pipeline, rather than by a declarative query over a shared model.

RDF and knowledge graphs. The Resource Description Framework (RDF) [7] represents relationships between entities identified by globally unique IRIs, organized as *knowledge graphs* [8]. For variant data it offers (i) linkage to external biomedical resources, following Linked Data practice [9–12], (ii) declarative querying with SPARQL [13], and (iii) access policies stated in a W3C standard, the Open Digital Rights Language (ODRL) [14], beside the data they govern. A user could then ask, in one query, which variants in a gene are associated with a phenotype and are accessible under a given policy. Getting there requires *semantic lifting*: a vocabulary that describes VCF’s concepts, and a systematic, reproducible mapping into it. The mapping is not trivial, because the VCF specification permits much heterogeneity [1]: INFO fields hold semi-structured annotations, FORMAT columns vary across datasets, and multi-sample files aggregate genotypes within shared records.

Related work. Earlier converters include the *rdf-vcf* reader and the web-based *VCF2RDF*, an isomorphic mapping to RDF/JSON-LD [17, 21]; a 2013 BioHackathon *vcf2rdf* using FALDO and tool-specific terms [18]; and JVarkit’s *VcfToRdf*, TOGO-var’s Rust *VCF2RDF* and SPARQLing Genomics’ *vcf2rdf* [4, 15, 16, 25]. General

Table 1 VCF-to-RDF converters and related semantic genomic-variation workflows, compared on the capabilities this study targets. Assessments are based on each system’s publicly available documentation. This is a feature comparison, not a measured one: no other converter was run on the same input here, so the table shows what each system documents, not that one converts better or faster.

Tool / workflow	CLI	RDF model	Reprod. workflow	Metadata + genotype	Extensible mappings	RDF compr.	Public release
<i>Dedicated VCF-to-RDF converters</i>							
VCF-RDFizer (this study)	✓	✓	✓	✓	(✓) ^c	✓	✓
TogoVar VCF2RDF [4]	✓	(✓)	✓	(✓)	(✓)	–	✓
JVarkit VcfToRdf [15]	✓	(✓)	✓	(✓)	–	–	✓
SPARQLing Genomics [16]	✓	(✓)	✓	✓	(✓)	–	(✓)
VCF2RDF (web) [17]	(✓)	✓	(✓)	(✓)	–	–	(✓)
BioHackathon vcf2rdf [18]	✓	(✓)	(✓)	(✓)	–	–	(✓)
<i>General genomic-format tools</i>							
BioInterchange [19]	✓	✓	✓	(✓)	–	–	✓
bio-vcf [20]	✓	–	✓	(✓)	✓	–	✓
ruby-rdf/rdf-vcf [21]	✓	–	(✓)	(✓)	–	–	✓
<i>Knowledge-graph construction pipelines</i>							
VariantKG ^a [22]	(✓)	(✓)	(✓)	✓	–	–	(✓)
AgroLD VCF2RDF ^b [23]	–	(✓)	–	–	–	–	(✓)
<i>Virtual semantic access</i>							
Semantic Beacons [24]	–	(✓)	(✓)	(✓)	✓	–	(✓)

✓ supported and documented; (✓) partial or limited; – not found in public documentation. **CLI**: command-line use in pipelines. **RDF model**: documented vocabulary for the output. **Reprod. workflow**: documented, reproducible VCF-to-RDF route. **Metadata + genotype**: header metadata kept with genotype and sample data. **Extensible mappings**: rules editable without changing the converter. **RDF compr.**: integrated compressed RDF output. **Public release**: distributed package, executable or container. ^aConverts VCF through SPARQLing Genomics’ vcf2rdf after SnpEff annotation, then adds its own ontology for CADD scores and patient metadata. ^bNamed in [23] as part of the AgroLD ETL; ratings reflect what its public repository showed in September 2026. ^cRML maps the record backbone, about 5% of the triples; the content-bearing layers (samples, INFO, QUAL, header) come from the wrapper’s emitters, configured by options rather than mappings (Section 2.2).

tools also reach RDF: BioInterchange emits GFVO from GFF3, GVF and VCF 4.1–4.2 [19], and *bio-vcf* serializes through templates [20]. Pipelines such as VariantKG build on these converters [22]; AgroLD names a VCF2RDF component whose code could not be located [23]; and Semantic Beacons exposes Beacon API responses as RDF through RML without materializing a file [24]. Table 1 compares what these systems document; measuring conversion cost and fidelity against them on a shared input remains to be done.

Variant data need not be lifted to RDF to be queried or joined. Analysis frameworks and variant stores such as Hail [26], GEMINI [27] and TileDB-VCF [28] join variants to annotation sources at cohort scale; GA4GH Beacon [29] and htsgget [30] serve variant discovery and regional slices over standard APIs; and HL7 FHIR Genomics [31] carries variants into clinical records. For an analysis run inside one of these systems, they are usually the better tool. What an RDF graph adds is a shared, schema-independent model rather than speed: identifiers shared with other resources, so that variants join external knowledge without a bespoke schema; access policies stated in a W3C standard beside the data they govern; and federation with existing SPARQL resources.

Section 4.2 measures the first two against a bcftools pipeline; federation remains future work.

Open challenges. Three gaps remain. First, converters differ in how explicitly they define their output’s semantics [4, 15, 17–19], and models such as GA4GH VRS [32], FALDO [33], the Sequence Ontology [34] and GFVO describe variation and locations but not the structure of a VCF document itself. Second, transformation rules are usually coupled to code or templates [4, 20], whereas declarative mappings such as RML [35], run by engines such as RMLStreamer [36], keep them inspectable and reusable. Third, lifting VCF to RDF multiplies its size: gzip and Brotli [37] must be undone before querying, whereas HDT [38] and COTTAS, a columnar triple table in Apache Parquet [39], compress while keeping triple-pattern access.

Contributions. This paper presents VCF-RDFizer (<https://github.com/ecrum19/VCF-RDFizer>) and asks whether its output is faithful, what it costs, and what it lets a user do. The VCF Core vocabulary it targets is the subject of a companion paper, in preparation [40]; here it is the target model. The contributions, each with the evidence for it, are:

1. **Faithful, verified conversion:** VCF to VCF Core RDF, with a validation layer that pairs each SPARQL answer with a VCF parser’s and applies SHACL shapes, and that is itself tested by fault injection (Sections 2.3 and 4.1).
2. **Linked, governed variant data across files, on real genomes:** plug-ins that link variants to shared identifiers and attach ODRL consent policies, evaluated on a clinical question over real genomes against a conventional bcftools pipeline, at three scales up to a whole genome (Sections 2.4 and 4.2).
3. **Cohort-scale modelling, measured:** two sample profiles, and why their triple counts mislead about their size (Sections 2.1 and 4.3).
4. **A cost model with guidance:** what conversion, HDT, COTTAS and SPARQL cost, measured to 657 million triples against a VCF parser and against indexed VCF access, and when each pays (Sections 4.4 and 5).

Table 2 states, for each VCF component and capability, what the release supports and what this study measured.

Table 2 VCF-RDFizer capability status and benchmark scope. “Supported” describes the release; the evaluation column states what this study measured, so that support is not mistaken for evidence. Tables and sections prefixed S are in the supplementary material.

VCF component / capability	Release	Evaluation	Notes / limitations
Core record fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported (v3.1.0)	Validated	Q1–Q4 exact-equal against cyvcf2/bcftools in every completed validation (Section 4.1)
QUAL	Supported	Q11 record digest	Typed literal for every record; a missing QUAL is <code>vcfc:Null</code>
INFO	Supported; structured by default	Q11 and Q12 digests	Typed value per record and key, per-allele items for Number=A/R/G, plus the complete <code>infoRaw</code> string
FORMAT and sample/genotype values	Supported	Q5–Q6 exact; Q13 digest	Half-calls, haploid calls and ploidy 4/3 survive conversion (Table S3); called allele, sample index and phasing are checked by the shapes
HEADER metadata	Supported	Census queries and shapes	Header attribute values are checked by the shapes rather than by a comparison query
Multi-sample conversion	Supported	Measured, 1–2,504 samples	Condensed adds 5 triples per sample; expanded adds 25 per sample per record and is not tractable at 2,504 samples (Section 4.3)
Structural and symbolic alleles	Lexically supported	HGSVC2 converted; Q2 passes	ALT remains the source lexical field; allele-value mutations are caught only by the full shape profiles
Queryable HDT, COTTAS	Supported	Measured	118 HDT and 34 COTTAS artifacts, all decode checks passed; COTTAS smallest throughout (Section 4.3)
Gzip/Brotli packaging	Supported	Covering set (S3)	Packages must be unpacked before querying; no codec comparison
Policy annotations	Plug-in v0.2.0 with v3.3.0	Real genomes, up to a whole genome	ODRL on files and declared graph selections, DUO purposes; every view checked against a VCF-side oracle; simulated consents, single-sample files; governed release, not anonymization (Section 4.2)
External linking	SPDI, Ensembl-gene and MyVariant linkers with v3.3.0	Real genomes	SPDI links every eligible call and joins genomes to ClinVar across contig conventions; tier 3 confirmed 92–93% of rsID rewrites (Section 4.2)

2 Implementation

VCF-RDFizer is a Python command-line tool, distributed through PyPI, Conda-Forge and Docker Hub under the MIT License, that converts VCF into RDF, builds optional compressed representations, validates its output, and hosts plug-ins for linking and policy (Figure 1). Its source, mappings and documentation are public; Supplementary Section S1 lists the pinned toolchain and the distribution and maintenance practice.

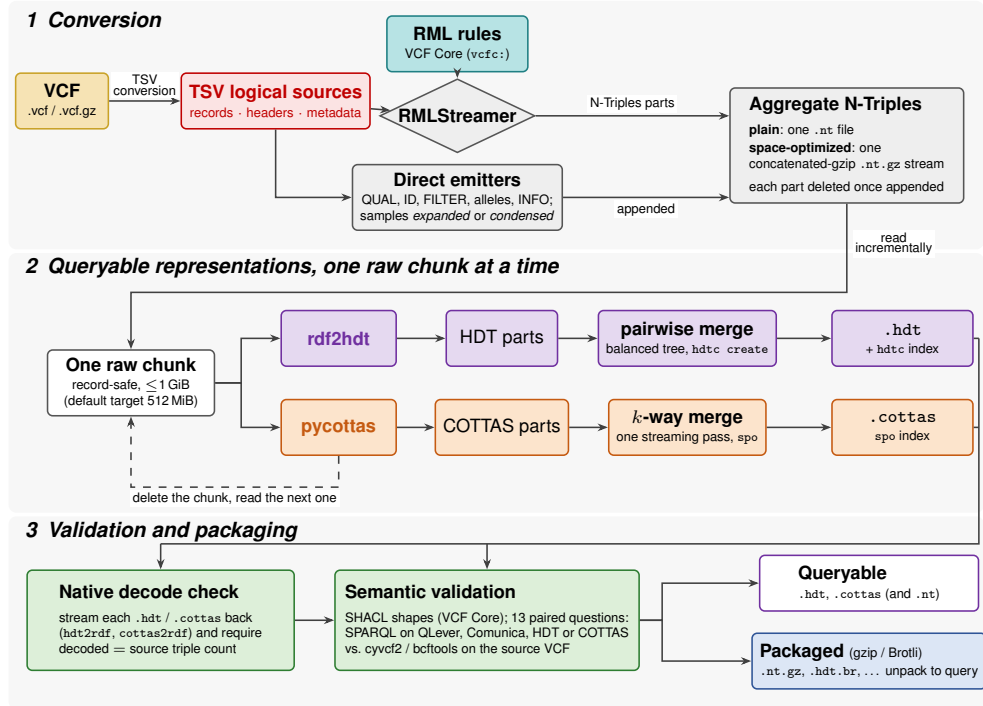


Fig. 1 VCF-RDFizer v3.1.0 pipeline. (1) **Conversion**: the VCF is split into three TSV logical sources; RMLStreamer applies the RML rules to emit VCF Core N-Triples, and direct emitters add the row-dependent fields RML cannot type per row (QUAL, ID, FILTER, alleles, structured INFO) and the samples in the *expanded* or *condensed* profile. Every part is appended to one aggregate and then deleted: a plain .nt file, or in space-optimized mode a concatenated-gzip .nt.gz stream. (2) **Queryable representations**: the aggregate is read incrementally and only one record-safe raw chunk exists at a time; **rdf2hdt** and **pycottas** each consume it before it is deleted. HDT parts are merged pairwise in a balanced tree with **hdtc** and indexed; COTTAS parts are merged in one streaming *k*-way pass with an **spo** index. (3) **Validation and packaging**: each artifact is decoded back and its triple count compared with the source; the graph is then checked against the VCF Core SHACL shapes and against thirteen questions answered both by SPARQL and by a VCF parser. HDT and COTTAS files are queryable as written; gzip or Brotli packages are for transport and must be unpacked before querying.

2.1 Semantic model

VCF-RDFizer emits RDF in the VCF Core Vocabulary (**vcfc:**, <https://w3id.org/vcf-core/vocab#>) [41], a separately maintained vocabulary that models the VCF specification itself, version by version, for VCFv4.1 through VCFv4.5. Its terms align with established resources for genomic coordinates (FALDO [33]), variant typing (the Sequence Ontology [34]), genotype and sample constructs (HERO [42]), and features of the GA4GH VRS schema v2.0.1 [43], so that VCF-specific structure is kept without duplicating mature ontology terms. The model preserves three information layers: file and header assertions, variant-record assertions, and sample and genotype assertions (Figure 2). The vocabulary carries classes, properties, domains and ranges but

no OWL restrictions; value and cardinality constraints are SHACL shapes, which the validation layer applies (Section 2.3).

Sample-level data can be emitted in two profiles. In the *expanded* profile every sample value is its own resource, which makes genotypes directly queryable. In the *condensed* profile values are stored as sample-ordered vectors, reducing the number of RDF resources while requiring decoding for access to individual sample values. Both profiles keep the same file, header and record context. Let V be the number of variant records, S the number of samples, and F the number of FORMAT keys per record. For a uniform FORMAT layout, the expanded profile adds VS sample-call resources and VSF FORMAT-value resources, whereas the condensed profile adds V cohort matrices and VF value vectors. Excluding the structure common to both, and assuming bounded ploidy and FORMAT-value arity, the sample-layer triple counts scale as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

The condensed profile removes the per-sample factor from the repeated RDF structure, but still stores all VSF values inside vector literals, so its size in bytes keeps growing with S even though its triple count does not (Section 4.3).

Core entities are minted from deterministic IRI templates over the source file and record position, so identifiers are reproducible but file-scoped: the same variant in two files receives two IRIs. Identity across files is added by linking (Section 2.4); GA4GH VRS computed identifiers [43] could serve beside NCBI SPDI and are future work.

2.2 Conversion and representations

Conversion has five stages: (1) each VCF is split into record, header-line and file-metadata TSV logical sources; (2) modular RML rule sets [35], authored in Turtle and executed by RMLStreamer [36], map them to RDF; (3) emitters add the remaining fields and the samples, and every part is appended to one N-Triples aggregate; (4) optional gzip or Brotli packages and queryable HDT and COTTAS representations are built from the aggregate; and (5) every representation is validated, packaged and reported. Options keep these concerns separate (`--rdf-storage-mode`, `--rdf-compression`, `--representations`, `--artifact-compression`), so an `.hdt` or `.cottas` file can be kept for querying while a `.gz` or `.br` package is made for transport.

RML maps the record backbone: nine triples per record, 5.3% of the HG005 graph’s triples. The content-bearing layers — samples, structured INFO, QUAL and the header, 94.7% of that graph — come from the tool’s emitters, configured by options, because RML cannot switch a literal’s datatype per row (a missing QUAL, for instance, is written `"^^vcfc:Null`). The TSV step exists because VCF is not a native RML logical source.

In **plain** storage mode the parts are appended to one uncompressed `.nt` graph; in **space-optimized** mode each part is gzip-compressed and appended as a member of one concatenated-gzip `.nt.gz` stream, which standard gzip readers expose as one N-Triples sequence. Either way each part is deleted once appended, so all parts and a full uncompressed aggregate never coexist.

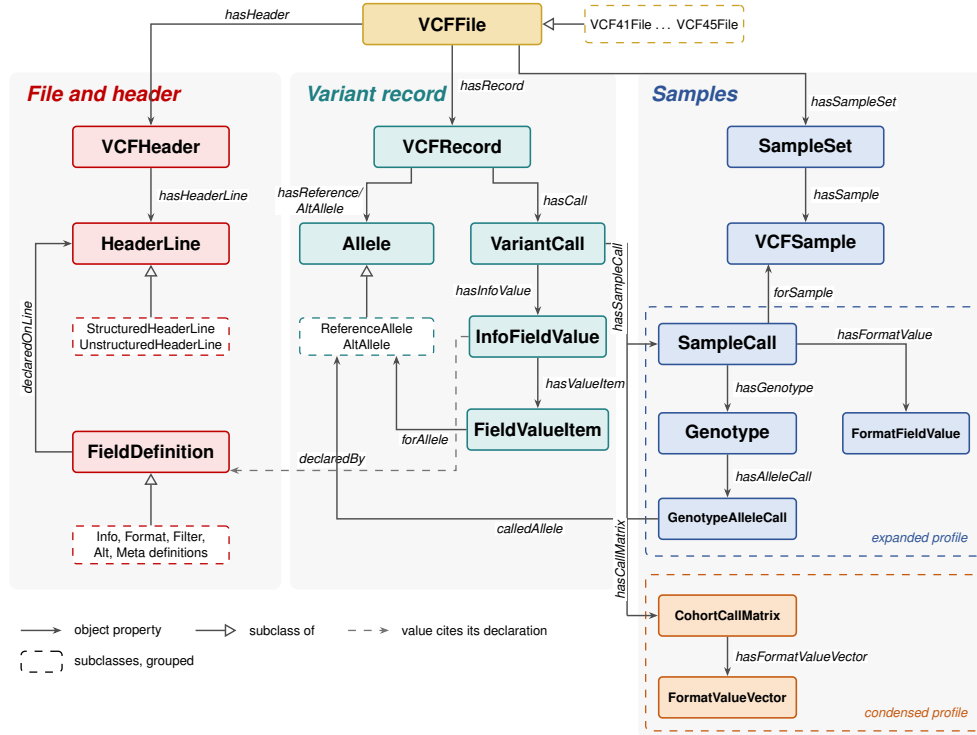


Fig. 2 VCF Core Vocabulary, simplified class diagram. The backbone of the model VCF-RDFizer v3.1.0 emits, drawn from the ontology bundle it ships; all class and property names carry the *vcfc:* prefix, <https://w3id.org/vcf-core/vocab#>. A *VCFFile*, typed with its version-specific subclass, links to three layers. In the **header** layer, header lines carry the INFO, FORMAT, FILTER, ALT and META definitions that values cite through *declaredBy*. In the **record** layer, each *VCFRecord* has ordered alleles and a *VariantCall* carrying the INFO values, whose Number=A/R/G items point at the allele they describe. In the **sample** layer, the call links either to per-sample *SampleCalls*, genotypes and allele calls (*expanded profile*) or to one *CohortCallMatrix* of sample-ordered vectors (*condensed profile*). Structural-variant, phasing, local-allele and value-typing classes are omitted; the full vocabulary, with worked examples, is documented at <https://ecrum19.github.io/vcf-core-vocabulary/>.

HDT and COTTAS are built one record-safe chunk at a time, so at most one uncompressed chunk exists while they are built. HDT parts are produced with *rdf2hdt* [38, 44], merged pairwise with *hdtc* [45, 46] and indexed. COTTAS parts are produced with *pycottas* [47] and merged in one streaming *k*-way pass with an *spo* index (v3.2.0 adds the other orders); COTTAS files are queried by a Comunica engine that evaluates SPARQL over the Parquet table with DuckDB [48]. Before the raw RDF is removed, every artifact is decoded with its native reader and its triple count compared with the source's; a mismatch fails the run. Supplementary Section S1 details the chunking.

Table 3 The thirteen-query equivalence workload. Each SPARQL result is normalized and must equal a result derived independently from the source VCF (Q4’s exact `FILTER` text via `bcftools query`). Ti/Tv, call rate and allele frequency are derived only after their underlying counts agree. Q11–Q13 count values in up to 256 buckets keyed on the first byte of SHA-256 over the IRI and values, which detects value changes that preserve ordinary counts but is not a collision-free proof of equality. For Q9, Q10, Q12 and Q13 the expected result follows the selected `INFO` and sample representations.

Query	Result compared, fields and scope
<i>Record level — every input</i>	
Q1 Record density	Records per contig and 1-Mb window (<code>CHROM</code> , <code>POS</code>); checks coordinates and integer positions
Q2 Allele shapes	Records per class — SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (<code>REF</code> , <code>ALT</code>)
Q3 Ti/Tv	Transitions, transversions and biallelic SNVs; single-base A/C/G/T alleles only
Q4 <code>FILTER</code> values	Records per exact <code>FILTER</code> string: <code>PASS</code> , <code>.</code> (not applied) or failed codes, combined codes kept whole
<i>Genotype level — inputs with sample columns and GT</i>	
Q5 Genotypes	Calls per sample and genotype class, including missing, haploid and other-ploidy calls; phased and unphased alike
Q6 Allele counts	Sites per recomputed (AC, AN) pair; single-ALT sites with complete 0/1 haploid or diploid calls
<i>Metadata and graph structure</i>	
Q7 File metadata	Exact <code>fileformat</code> , <code>reference</code> and <code>source</code> strings, with markers for absent declarations
Q8 Header census	<code>##</code> lines per header key (<code>INFO</code> , <code>FORMAT</code> , <code>FILTER</code> , <code>contig</code> , ...)
Q9 Predicate census	Triples per predicate against the counts expected from the VCF
Q10 Class census	Resources per <code>rdf:type</code> against the expected inventory
<i>Value preservation — IRI-bound digests</i>	
Q11 Record digest	Records per bucket over each record’s IRI, <code>CHROM</code> , <code>POS</code> , <code>ID</code> , <code>REF</code> , <code>ALT</code> , <code>QUAL</code> , <code>FILTER</code> and raw <code>INFO</code>
Q12 <code>INFO</code> digest	Structured <code>INFO</code> values per bucket, bound to record and key; empty in raw-only <code>INFO</code> mode
Q13 <code>FORMAT</code> digest	<code>FORMAT</code> values (expanded) or sample-ordered vectors (condensed) per bucket, beyond <code>GT</code>

2.3 The validation layer

Readable output is not faithful output. The validation layer (`--validate`) therefore asks the graph thirteen standard VCF inspection and quality-control questions [6, 49, 50] in SPARQL, and derives each expected answer independently from the source VCF with `cyvcf2`, using `bcftools query` for exact `FILTER` text (Table 3). Fourteen graph-integrity preflight queries run first: syntax, cardinality of core record properties, datatypes, hashes, versions and counts.

Results must be exactly equal after normalization: strings for Q7 and integer counts elsewhere, including the digest buckets of Q11–Q13, which extend coverage to `QUAL`, `INFO` and `FORMAT` without guaranteeing value-by-value equality. The comparator keeps contig and sample identifiers, distinguishes `PASS` from `.`, and reports missing, extra and differing rows. Q5–Q6 apply only when the source has a `GT` field, and each input is validated only against its own graph, so inputs on different assemblies are never compared.

The same SPARQL text runs under four engines: QLever [51], the default; Comunica over N-Triples (`@comunica/query-sparql-file`) [52, 53]; Comunica over HDT (`@comunica/query-sparql-hdt`) [54]; and the DuckDB-backed COTTAS engine [48].

Any engine can be pointed at any artifact, and `--validation-engine all` runs all four and records whether they agree. The layer also applies the vocabulary’s SHACL shapes in three profiles. The default profile checks cardinality and datatypes; the other two check uniqueness and value agreement, and are costlier (Section 4.1).

What this establishes is *representation equivalence* for the stated fields: a SPARQL engine and a VCF parser derive the same deterministic summaries from two representations of the same source. It does not establish that the source calls are biologically correct, normalized or concordant with a truth set.

2.4 Linking and policy plug-ins

Two host-side commands extend a converted graph without changing it.

Linking. `vcf-rdfizer-link` writes a separate N-Triples linkset, so linking never alters the converted graph or its validation. A linker is declared in Turtle at one of three tiers. *Tier 1* rewrites tokens the file already holds, such as rsIDs, into IRIs, without checking them. *Tier 2* joins against a reference pinned by digest and assembly. The `spdi` linker trims each ALT allele to its minimal form and emits its NCBI SPDI expression [55] on the RefSeq accession as an IRI, so a variant receives the same IRI in every file, 17 or `chr17`; `ensembl-genes-grch38` links each call to the Ensembl 116 genes its REF span overlaps, with contig aliases. *Tier 3* confirms identifiers against a live service through a short Python resolver: requests are batched under a declared rate and budget, responses are cached with digests, and any failure fails the linker closed, publishing nothing. `rsid-myvariant` confirms rsIDs against MyVariant.info [56]. When the input is RDF, the join fields are read with SPARQL, from an endpoint or from a temporary Oxigraph store.

Policy. `vcf-rdfizer-policy` attaches ODRL policies [14] to graphs and derives one release view per request in three generic steps, each configured in Turtle rather than code. A rule’s target is a resource, or a selection computed by a declared SPARQL selector; region, variant and linked selectors ship with the tool, and others can be declared in a policy file. A profile’s ownership rule states what a withheld resource takes with it: under the VCF Core profile, withholding a record withholds its call, alleles and sample calls, and withholding a file withholds its header too. ODRL semantics then decide, with deny winning and anything no permission covers withheld; purposes come from any RDFS or SKOS vocabulary, by default a subset of the GA4GH Data Use Ontology (DUO) [57]. Selector parameters are injected as an inline `VALUES` block opening the query’s outer group, because SPARQL joins a trailing `VALUES` clause only after the group is evaluated: a region selector written that way selects nothing, and a prohibition on the region fails open. Version 0.2.0 evaluates against a SPARQL endpoint and streams each view from the N-Triples, byte for byte and in input order. Every view is then checked:

- its policy digest matches;
- nothing a binding prohibition owns is present, and nothing lacks a permission;
- no reference dangles, a question put to a second endpoint that serves the view;
- its records equal an oracle that evaluates the same policy over a graph built directly from the VCF text, so an extra record is a leak and a missing one over-withholding.

The plug-in provides governed release, not anonymization: a released genotype remains identifying, views keep the original IRIs, and obligations are recorded but not enforced.

3 Evaluation design

3.1 Inputs and experiments

The ten public VCFs (Supplementary Table S2) span single-sample genomes, a large phased cohort and sequence-resolved structural variants: five Personal Genome Project donations [58], one HG002 precisionFDA/Genome in a Bottle challenge file [59], the 2,504-sample 1000 Genomes Phase 3 chromosome-20 call set on GRCh37 [60], the 32-sample HGSVC2 structural-variant call set [61], and the HG004 and HG005 Genome in a Bottle v4.2.1 benchmark files [62]. The corpus experiment converts the first 250,000 records of each input, with complete headers, except HG005_GRCh38, which is converted whole; all cells use space-optimized storage and build both HDT and COTTAS. The 1000 Genomes file is excluded from the expanded arm, which withholds any input with more than 1,000 sample columns, and is exercised through 10,000-record slices instead.

Record scaling uses HG005_GRCh38 at 10,000, 100,000 and one million records (three replicates each) and whole. Sample scaling holds 10,000 records of 1000G_phase3_chr20 fixed and varies the samples from 1 to 2,504 under both profiles, with INFO fixed, isolating Equation 1. Paired runs compare the storage modes. Fixtures cover VCF versions 4.1–4.5 and eleven awkward inputs (Supplementary Section S2), and further cells cover every configuration value, each operating mode and a memory sweep (Supplementary Section S3).

Validation. The full paired workload runs on the fixtures, on a 10,000-record multi-sample fixture under both profiles and all three artifacts, and on a 100,000-record slice of HG005_GRCh38; the corpus cells receive native decode and triple-count checks only. A mutation experiment injects 113 targeted corruptions into a graph and counts how many the layer detects, with and without the shapes. Determinism is tested by converting one input twice, by a compress–decompress round trip, and by re-indexing an indexed artifact.

Retrieval. Query-cost cells run without shapes, three replicates from a cold start, with identical scope on both sides; setup is reported apart from query time, representation build is excluded, and answers are compared before any timing is read. A 1,000,000-record slice (170,935,101 triples) and the whole genome (657,425,805) test the costs at scale. A regional experiment asks five region-restricted questions over 1 kb to 10 Mb windows (twenty per size, three replicates) against tabix-indexed bcftools and cyvcf2 and an unindexed scan. A tabix seek also returns records that start before the window and overlap it, so every VCF arm drops out-of-window POS and all arms answer the same question.

3.2 The use case

To test linking and policy on real data, we asked one clinically recognizable question of real genomes by two routes that share no code: *which participants carry a ClinVar-classified variant in one of the 81 genes of the ACMG secondary-findings list, SF v3.2*

[63], and what may each of three requesters see? The RDF route converts every input with VCF-RDFizer, links each with the `spdi` and `ensembl-genes-grch38` linkers, evaluates the policies with `vcf-rdfizer-policy` against QLever [51], and asks one SPARQL query per requester over that requester’s view and ClinVar. The conventional route renames contigs to ClinVar’s naming, annotates each genome with ClinVar’s `CLNSIG`, `CLNREVSTAT` and `GENEINFO` using `bcftools` [6], and applies the consents in a Python script. Both read one case definition: the question, the ClinVar review statuses that count (one star or more), and the consents. A gate compares the two carrier lists for every requester, keyed by participant, gene, contig, position and alleles, and no timing is reported unless they are identical.

The genomes are real and public; the consents are simulated, and every output says so. Each file carries a consent stated as DUO purposes; a withdrawal is a prohibition, so deny wins. One cohort rule releases records in the list’s 28 cancer-predisposition genes only for clinical care. The requesters are a clinical genetics laboratory (clinical care, CC), a cardiovascular consortium (disease-specific research, DS) and a biobank researcher (general research use, GRU).

Three arms separate integration, breadth and scale. Arm 1 is five genomes from three sources: two Personal Genome Project genomes [58], HG004 and HG005 from Genome in a Bottle [62], and an HG002 precisionFDA submission whose contigs are named 1, 2, ... rather than `chr1`. Arm 2 is 104 unrelated participants of the 1000 Genomes high-coverage call set [64], four from each of its 26 populations, with seeded simulated consents, four of them withdrawn. Arms 1 and 2 are restricted to the 81 genes’ spans (7.1 Mb, Ensembl 116) and normalized with `bcftools` (left-aligned, multi-allelic records split). Arm 2 drops the panel’s INFO fields, which describe all 3,202 panel samples rather than the participant and made up 95% of each participant’s triples; the frequencies the question uses are recorded once, in a sites-only file. Arm 3 is HG005’s whole genome through the same pipeline, so its answer must equal its arm-1 answer. ClinVar’s 2026-09-13 release [65] is restricted to the same spans.

Effort. On arm 1 we counted the lines each route’s author writes, and the lines four changes add and remove: a withdrawal, a requester with a new purpose, a cardiac panel in place of the cancer genes, and a rule on one variant. Each change was applied to both routes as a real edit. Both edited routes then had to release the same records to every requester on a fixture with records in a cancer gene, in a cardiac gene, in neither, and at HFE C282Y.

3.3 Execution

The conversion, validation and retrieval campaign ran as 143 recorded cells, 96.2h of wall time, all produced by the published v3.1.0 image on two identically specified 8-core, 33.6GB hosts. Each experiment ran on one host without concurrent jobs, so timings are compared within a host. Every cell records its command, host, tool commit and image digest, and no cell failed. The regional cells need a runner that postdates v3.1.0 and were produced by a later build. The use case converts with v3.2.0 and links and governs with v3.3.0’s plug-ins. Supplementary Section S1 gives the hosts, the image digest and commits, the host calibration and every cell of the campaign.

Table 4 Validation evidence, grouped by the question each layer answers. Counts are pass/total, derived from archived reports. The sensitivity block is what licenses the two above it: a suite that cannot fail proves nothing by passing.

Check	Passed	What it establishes
<i>Is the graph faithful to the source VCF?</i>		
N-Triples syntax (rapper)	76/76	The serialization parses
Paired SPARQL vs cyvcf2 (Q1–Q13)	984/988 [†]	Exact equality of normalized query results
Cross-check invariants	1,144/1,144	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	76/76	QLever, Comunica, HDT, COTTAS (25 with all four)
Native decode and triple count	152/152	118 HDT, 34 COTTAS; source = decoded
<i>Is the pipeline reproducible?</i>		
Determinism, two runs	1/1	Identical SHA-256; 17,098,746 triples, 0 duplicate lines
Compress and decompress	1/1	Lossless through the packaging path
Index idempotence	1/1	Re-indexing does not perturb the artifact
<i>Would a fault actually be noticed?</i>		
Injected faults caught, queries only	96/113 (0.850)	17 faults, spanning 10 mutation classes, are invisible to the queries
with the default shape profile	96/113 (0.850)	Cardinality and datatypes catch none of the 17
with all three shape profiles	113/113 (1.000)	Uniqueness and value agreement catch all 17; both controls still pass
SHACL shapes, default profile	58/58	Cardinality and datatype constraints, 0 violations

[†]Four comparisons were verified as not-applicable rather than passed; they are excluded from paired-comparison totals.

4 Results

4.1 Fidelity

Across the campaign the validation layer ran 78 times, on N-Triples, HDT and COTTAS artifacts under all four engines (Table 4). Its paired value-level comparisons ran on fixtures and slices of up to 17.1M triples; the nine corpus files and the 657M-triple genome received native decode and triple-count checks only. Of 988 paired comparisons, 984 were exactly equal and 4 were verified not applicable: the two genotype questions on each of the zero-record fixture’s two artifacts. None mismatched, and all 1,144 cross-check invariants held. Engine agreement is a real comparison only where several engines answered: in the 25 validations where all four did, every engine agreed, while in the other 51 completed validations a single engine answered. The native decode check passed on all 118 HDT and 34 COTTAS artifacts. The default shape profile ran on 58 validations and found no violations, at 0.8–396.8s each. Two conversions of the same input were byte-identical (17,098,746 triples), a compress–decompress round trip reproduced the same digest, and re-indexing left an artifact unchanged.

Sensitivity. The mutation experiment injected 113 targeted corruptions — wrong allele, wrong coordinate, wrong filter token, wrong file metadata, blank nodes, flipped phasing, corrupted sample index — and the layer with all three shape profiles detected all of them. The two halves contribute differently. The comparison queries alone detect 96 (0.850). The other 17 fall into ten classes the queries structurally cannot see: allele kind and value are counted but never compared with the raw ALT column, header attribute values and the derived contig count are counted but not read, and record index, sample index, called allele and phasing status have no oracle. The full shape set catches every one, and both unmutated controls pass in both profiles, so the shapes are not simply rejecting conformant graphs. But the default profile, which checks cardinality and datatypes, catches none of the 17: with it alone the layer detects 96 of 113, exactly the queries’ score. The two profiles that do catch them check uniqueness and value agreement, self-join the graph, and cost 33.7s and 73.2s on a 2,000-triple graph against 1.7s for the default; v3.1.0 therefore applies the default to every validation and the full set only to inputs up to 16 MiB, so on any larger input those ten classes go undetected.

Real defects. During development the layer caught three conversion defects that the record counts alone could not show. A header-only file omitted the samples its header declares, which the predicate and class censuses (Q9, Q10) detected. Expanded genotypes referenced allele resources that a raw-INFO conversion never emitted: a 1,000-record test held 1,972 such references, which the queries counted without following and the shapes reported as 5,916 violations. And in the policy plug-in, a withdrawn file’s header and sample set remained in every release view although every record decision was correct; the VCF-text oracle caught it, and it is why withholding is defined by ownership. All three are fixed. Supplementary Section S4 records these and the measurement and reporting defects testing found.

Inputs and encodings. Every declared version from VCFv4.1 to v4.5, and an undeclared one, converted; v4.4 and v4.5 fields such as local alleles are carried, not interpreted. Nine of eleven awkward inputs converted and validated, a truncated gzip was refused with a clear diagnostic, and a sites-only fixture could not be validated because bcftools rejects it (Supplementary Section S2). Every configuration value validated, shapes included, and all four engines agreed on both profiles, both storage modes and all three artifacts in sixteen validations (Supplementary Section S3).

4.2 A governed clinical question over linked real genomes

Linking. The `spdi` linker linked every eligible call, all but symbolic alleles, in every file, and so joined the genomes to ClinVar whatever each called its chromosomes (Figure 3, Table 5). On seven ClinVar *BRCA1* variants it reproduced NCBI’s SPDI exactly outside repeats; inside a repeat NCBI writes the whole repeat, and the trimmed form denotes the same allele. The live tier was tested on the two PGP genomes’ real rsIDs. Against Ensembl’s REST service one batch kept returning HTTP 500 after three backed-off retries, and the linker failed closed. Against MyVariant.info, 21 batched requests confirmed 9,223 of 9,951 and 9,479 of 10,316 tier-1 rsID links. Each of three unconfirmed rsIDs we looked up had been merged into another in dbSNP: tier 1, which rewrites whatever rsID a file holds, passes stale identifiers through unnoticed.

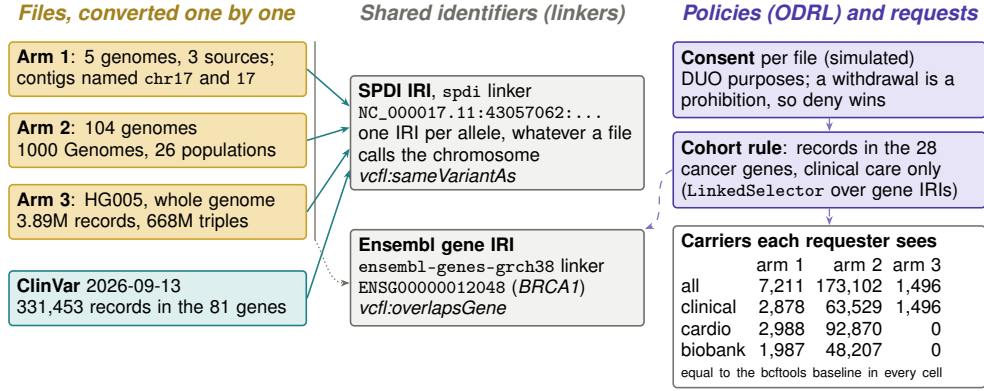


Fig. 3 A governed clinical question over linked real genomes (vcf-rdfizer-testing experiment 17). Each file is converted on its own and keeps its file-scoped IRIs. The **spdi** linker gives each allele the NCBI SPDI IRI of its RefSeq accession, the same in every file whatever the file calls the chromosome, so the genomes and ClinVar join on it; the **ensembl-genes-grch38** linker joins each genome's calls to the Ensembl genes they overlap. A consent per file and one cohort rule, selected through the gene links, decide what each requester sees: clinical is a clinical-care laboratory (CC), cardio a disease-specific cardiovascular study (DS) and biobank general research use (GRU). Every count equals the bcftools route's. Arm 3 is HG005, whose only consent is clinical care; its whole-genome answer equals its arm-1 answer.

Table 5 Linking on the use case's real inputs: calls linked, summed over the arm's files. Eligible means an explicit allele: symbolic alleles such as have no SPDI expression and no REF span. In arms 1 and 2 every record lies in a gene span, so every call has a gene.

Linker (tier)	Arm 1	Arm 2	Arm 3	ClinVar	Result
spdi (2)	51,302	1,149,198	3,887,200	331,413	Every eligible call; joins genomes to ClinVar across contig names; 87s for the whole genome
ensembl-genes-grch38 (2)	51,321	1,149,198	1,704,921	—	Each file reloads the 108 MB GFF3, 19 of arm 2's 22 min of linking
rsid-dbsnp (1)	20,267	—	—	—	The PGP genomes' rsIDs, rewritten unchecked
rsid-myvariant (3)	18,702	—	—	—	92–93% of the tier-1 links confirmed in 21 requests; the rest were merged rsIDs
rsid-ensembl (3)	0	—	—	—	A persistent HTTP 500; failed closed, no partial linkset

Answers. The two routes returned identical carrier lists for every requester in every arm (Table 6), and every release view passed all four checks. The reportable subset, the Pathogenic and Likely pathogenic classifications a secondary-findings report acts on, is counted rather than filtered on: it is empty in arms 1 and 3 and holds 8 carriers in arm 2. The question covers every classification because the five arm-1 genomes carry no reportable variant in the 81 genes, which is itself an answer. HG005's whole-genome answer equals its arm-1 answer key for key, 1,496 carriers. Arm 2 also asks

Table 6 Carriers each requester may see, identical between the RDF route and the bcftools route in every cell: all (no policy), clinical care (CC), the cardiovascular disease-specific study (DS) and the biobank’s general research use (GRU). Triples are the genomes’ graphs, before links. *Rare*: carriers of variants with panel AF < 0.01, a join across the genome, ClinVar and the panel’s frequency file.

Arm	Files	Records	Triples	All	CC	DS	GRU
1: five genomes	5	51,386	9.0M	7,211	2,878	2,988	1,987
2: cohort	104	1,150,288	70.3M	173,102	63,529	92,870	48,207
rare in the panel				4,154	1,621	2,198	1,144
3: whole genome	1	3,887,810	668.1M	1,496	1,496	0	0

Table 7 Cost of the RDF route per arm, on 8-core, 31 GB machines. Triples are those in the unrestricted query’s index: the genomes, ClinVar and all links. Convert and link are totals over the arm’s files. View and check are per requester: writing the view, then checking it, which for a non-empty view includes building its index. Index build and query are per requester too: one cold run of the carriers query over that requester’s view, ClinVar and the links, after the index is built.

Arm	Triples	Convert	Link	View	Check	Index build	Query
1	42.5M	3.3 min	1.4 min	32–39 s	—	2.2 min	172–181 s
2	110.3M	31 min	22 min	59–80 s	3–5 min	2.4–3.3 min	176–202 s
3 (clinical)	701.5M	53 min	7.5 min	38 min	68 min	45 min	205–209 s
3 (empty views)				28 min	1 min	2.2 min	<1 s

a three-file question: the panel’s allele frequencies, recorded once in a sites-only file of the 68,635 sites the cohort carries (6.5M triples, in place of about 850M triples of panel INFO copied into every participant), are joined through the same SPDI IRIs to return the carriers of variants rare in the panel (AF < 0.01). The baseline looks up the same frequency by position and alleles, and the two agree for every requester.

Cost. Query time barely grows with scale (Table 7): a cold carriers query took 172–181 s in arm 1, 176–202 s in arm 2 and 205–209 s over arm 3’s 701M-triple index, because the ClinVar side of the join dominates it. Building the index is what scales, from about 2 minutes to 45. Governance was first bound by Python rather than QLever: in arm 2 each view took 20–27 min to write, because every line was checked against each of 109 rules. Merging the binding rules’ selections once and filtering the inputs in parallel cut that to 59–80 s, with byte-identical views. Arm 3’s clinical view, every record released, streamed in 38 min on two workers, one per input file. Arm 3 peaked at 13.1 GB of memory and 34 GB of disk.

Effort. The RDF route’s rules are 72 lines, 41 of policy and 31 of query, against 131 lines of script; its linkers, selectors and profile ship with the tool (Table 8). Three of the four changes are data edits of similar size in both routes, because the baseline was written to read its rules from a table. They differ where a change introduces a new kind of rule. Withholding one named variant except for clinical care needs three new lines of code in the baseline, and six lines of policy and no code in the RDF route. Stated on the variant’s SPDI identity, the rule also holds for HG002, whose file writes the contig as 6; a selector matching the contig string would have silently missed it. All four edited routes still agreed on the fixture.

Table 8 What each route’s author writes (arm 1) and the lines four changes add and remove. Blank lines, comments and docstrings are not counted. After each change both routes were re-run on a fixture and released the same records to every requester. POA is DUO’s population-origins-or-ancestry purpose; it sits outside general research use, so a GRU consent does not cover it.

	RDF route	bcftools route
Rules written	72: policy 41, query 31	131: shell 33, Python 98
Data written	28 gene IRIs; the query’s gene list (1 line)	consent and requester tables (100 lines); gene list (81)
Linkers, selectors, profile	shipped with the tool	—
A participant withdraws	policy +2/ − 1	data +2/ − 1
A requester with a new purpose (POA)	purpose vocabulary +2	data +4
The panel becomes 40 cardiac genes	policy +40/ − 28 (gene IRIs)	data +41/ − 29 (gene symbols)
One variant (HFE C282Y) clinical care only	policy +6, no code	code +3, data +9/ − 1

4.3 Conversion cost: scaling, storage and representations

Records. On the HG005_GRCh38 ladder (Supplementary Figure S2a), triple production is exactly linear in records (fitted exponent +0.999, 95% CI [+0.991, +1.007]), at a constant 170.9–171.9 triples per record. Final artifact bytes scale at +1.033, peak workspace at +1.025 and end-to-end wall time at +1.064 (40.7 s, 423.7 s and 5,476.0 s at 10 k, 100 k and 1 M records; replicate sd ≤ 11.0 s). Peak resident memory of the mapping stage stays between 1.0 and 1.7 GB across the hundredfold change: the streaming design is memory-bounded, and it is disk, not RAM, that scales with the data. The RMLStreamer mapping stage accounts for only 12.2–41.4 s of those runs; building the queryable representations dominates.

Samples. Re-emitting the same 10,000 records against 1 to 2,504 samples separates the profiles as Equation 1 predicts (Figure 4). Both series are exactly linear: $1,359,498 + 250,005 S$ triples expanded (25 per sample per record, plus 5 per sample of declarations; the fitted exponent of +0.795 reflects the fixed record layer at low S) and $1,439,498 + 5 S$ condensed, so at 2,504 samples the expanded graph has 627,372,018 triples, $432\times$ the condensed one. Triple counts mislead about size. Condensed values move into vector literals, so the condensed HDT grows $5.8\times$ (10.2 to 59.2 MB) while its triples grow 0.9%, and the byte gap at 2,504 samples is $322\times$ in gzip-framed N-Triples and $69\times$ in HDT. The condensed HDT is smaller than the uncompressed VCF only from 256 samples (10.2 against 1.5 MB at one sample; 59.2 against 101.6 MB at 2,504); the expanded HDT is $40\times$ the VCF (4.10 GB) and took 15,269 s to build end to end, against 250 s condensed. Extrapolated to the full 1,812,841-record call set, the expanded profile would need roughly 2 TB against a 189 GB volume, so the tool refuses such a run and names the condensed profile.

Storage mode. Plain and space-optimized storage give identical triples and the same final artifacts, to within one 4 kB filesystem block, and differ only in transient disk (Supplementary Figure S2b). Space-optimized mode cuts peak host

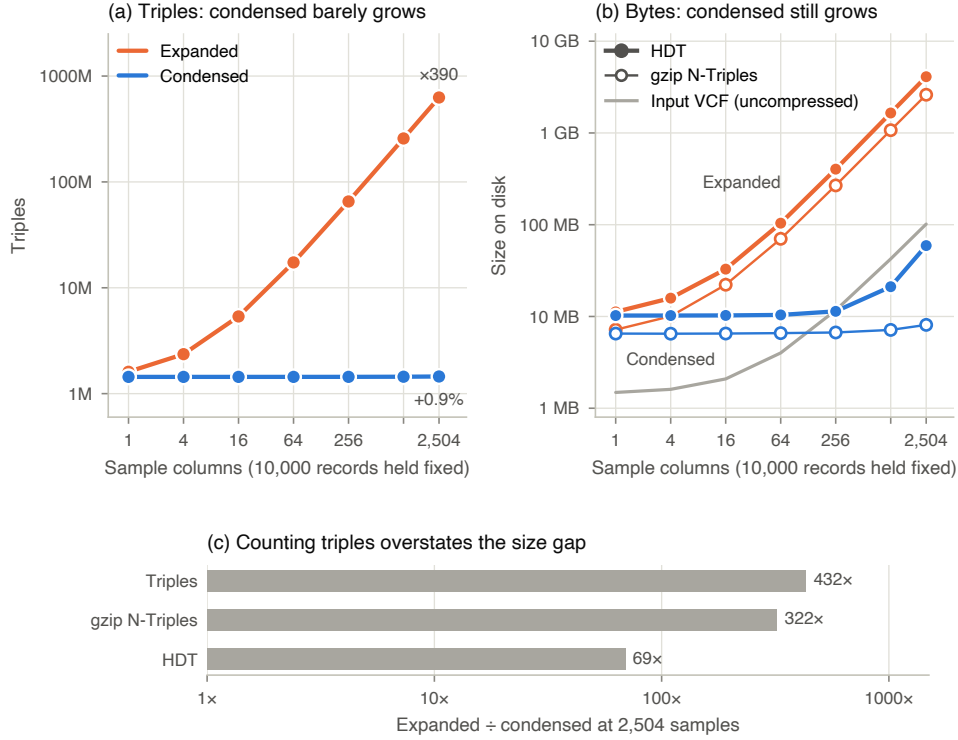


Fig. 4 Triples against bytes for the two sample profiles. The same 10,000 1000G_phase3_chr20 records re-emitted against 1 to 2,504 sample columns (the unlabelled tick is 1,024); medians where the rung was replicated. (a) Triples. The expanded profile adds resources for every sample call; the condensed profile adds five triples per sample and keeps each record's per-sample values in sample-ordered vector literals. (b) Size on disk of the two stored artifacts, HDT (filled) and the gzip-framed N-Triples that space-optimized mode writes (open), with the uncompressed input VCF for reference. The condensed artifacts grow although their triple count barely changes, because the vectors lengthen with every sample. (c) The expanded-to-condensed ratio at 2,504 samples, measured three ways. Panel (a)'s $\times 390$ is a different ratio: the expanded graph's growth from 1 to 2,504 samples (627.4M over 1.61M triples), where (c) divides expanded by condensed at the same sample count (432 $\times$).

workspace by $9.19\times$ (2.98 GB to 0.325 GB) on the 100k-record input and by $7.68\times$ (43.40 GB to 5.65 GB) on **test-larger**, a 1,155,741-record, single-sample prefix of the NG131FQA1I genome, at a time cost of $+5.2\%$ and $+3.2\%$. On the replicated input the mapping stage is equivalent within a $\pm 10\%$ margin fixed before the run ($+0.0\%$, 90% bootstrap CI $[-1.9\%, +0.4\%]$).

Representations. Across the corpus (Figure 5), COTTAS is consistently the smallest artifact: $0.37\text{--}0.55\times$ the stored gzip-framed N-Triples and $0.27\text{--}0.37\times$ the HDT built from the same graph. Uncompressed HDT is *larger* than the gzip-framed N-Triples it was built from, by $1.36\text{--}1.82\times$. COTTAS takes $1.5\text{--}3.0\times$ longer to build than HDT (31,230s against 20,911s on the whole HG005_GRCh38). Peak workspace is $4.7\text{--}30.3\times$ the uncompressed input. The whole genome, 657.4M triples, took 16.07 h

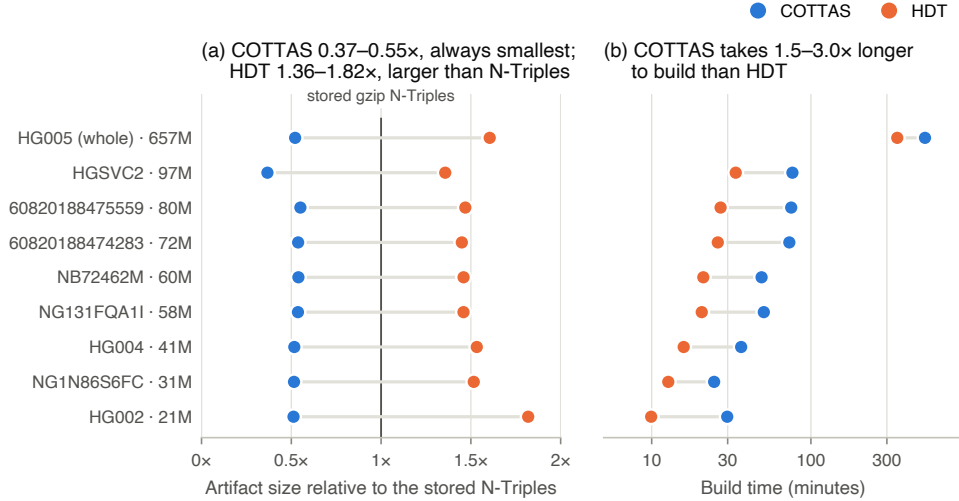


Fig. 5 Queryable representations in the corpus experiment, one run per completed input, ordered by graph size (triples after each label). Eight files use their first 250,000 records with headers retained; HG005_GRCh38 is processed whole. The 2,504-sample 1000 Genomes cell is guard-skipped in expanded mode. Dataset labels are abbreviated; full names are in Supplementary Table S2. (a) Artifact size relative to the stored gzip-framed N-Triples that space-optimized mode writes. (b) Build time of each representation from the same graph, on a log scale. Every artifact passed the tool’s native streamed decode and source-versus-decoded triple-count check.

end to end, of which the mapping stage was 146.8 s and the two representation builds 14.5 h; its final outputs (HDT 4.63 GB, its index 4.74 GB, COTTAS 1.50 GB) were 10.88 GB, 78× the 139 MB download, and the gzip-framed N-Triples alone 2.89 GB.

4.4 Retrieval cost

At fixture scale, the engine sets the cost and the artifact does not. On the 17.1M-triple graph, per-question times from N-Triples, HDT and COTTAS differ by under 5% for the same engine (Figure 6c). On the 0.96M-triple graph the thirteen questions cost a mean of 1.1 s under QLever, 23.4 s under Comunica, 38.2 s under the COTTAS engine and 45.1 s under the HDT-backed engine (Figure 6b), all four agreeing in every run.

Against a VCF parser, the answer depends on the workload. For a *single* question on the 17.1M-triple graph, cyvcf2 must scan the VCF and costs 11.5 s whichever question it is, while QLever answers in 0.005–4.62 s once its index is built: 2.5× to 2300× faster (Figure 6a). That index takes 22.8 s, so the parser wins any single cold question. For the *whole* thirteen-question batch, cyvcf2 answers everything in one 12.4 s pass, against 17.1 s of QLever query time plus the 22.8 s index.

At whole-genome scale, QLever’s cost is linear in triples. The thirteen questions cost 1.15, 0.99, 0.96 and 0.97 s per million triples at 0.96M, 17.1M, 171M

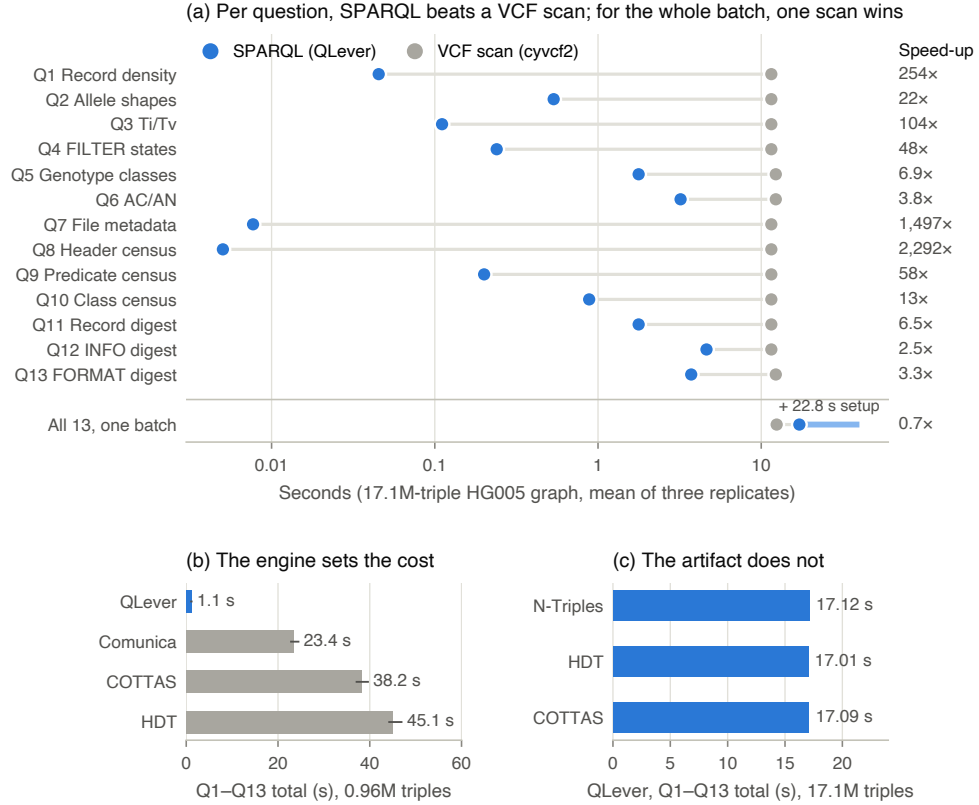


Fig. 6 Retrieval cost. Every comparison returned **PASS** before any timing was compared, and representation-build cost is excluded throughout. (a) Paired per-question cost on the 17.1M-triple HG005_GRCh38 graph (100,000 records), mean of three replicates. The cyvcf2 cost is that of answering the one question from the VCF, which requires a scan whichever question it is. QLever’s one-time setup is excluded from the per-question rows and drawn on the batch row, where one parser pass answers all thirteen questions. (b) The same thirteen questions under each engine on the 0.96M-triple graph: mean of nine runs (three artifacts in each of three replicates), with the range. All four engines agreed in every run. One-time setup, not included: QLever 2.0–2.1 s, Comunica (@comunica/query-sparql-file) 11.7–12.5 s, the DuckDB-backed COTTAS engine 6.6–39.2 s, and the HDT-backed Comunica engine (@comunica/query-sparql-hdt) 6.1–11.3 s. (c) QLever on the 17.1M-triple graph from each artifact, mean of three replicates; setup 22.7–23.0 s in each case.

and 657M, and the index 1.27–1.35 s per million above the smallest graph (Supplementary Table S4). On the whole genome they took 634.6 s after an 868.0 s index build, replicates within 0.7%, every answer equal to the parser’s. The artifact still does not matter (634.6, 636.3 and 636.6 s from N-Triples, HDT and COTTAS); only the one-time decode does (179–192, 955–972 and 592–601 s). Neither Comunica-backed engine completes the workload at 171M triples, even with a 24 GB heap: each answers five questions, then the COTTAS engine fails to allocate and the HDT engine reaches the one-hour ceiling. On those five the COTTAS engine is 16.7× faster, where at 0.96M

Table 9 Regional retrieval on the 17.1M-triple HG005_GRCh38 graph (100,000 records). Median over the five region questions of each question’s median across twenty windows and three replicates, in milliseconds. Every arm answers the same POS-based window; across both graphs, 11,160 executions produced 0 failures and 0 disagreements. One-time setup, excluded: QLever index 23.2s; bgzip plus tabix 0.39s.

Arm	1 kb	100 kb	1 Mb	10 Mb	Cost shape
QLever (SPARQL)	9.6	10.0	9.6	11.1	Flat: 1.16× over a 10,000× window range
cyvcf2, indexed	3.3	4.8	14.9	117.8	Grows with records returned (35.7×)
bcftools, indexed	4.3	4.8	7.5	32.7	Grows with records returned (7.5×)
cyvcf2, unindexed scan	635.2	638.0	632.0	649.9	Flat: reads the whole file regardless

the two were within 1.2×, so engine orderings at fixture scale do not predict behaviour on a cohort.

Against indexed VCF access, the two cross over. Across 11,160 regional executions the arms recorded 0 failures and 0 disagreements (Table 9). On the 17.1M-triple graph QLever’s cost is almost independent of window size, 9.6 ms at 1 kb and 11.1 ms at 10 Mb, because its index is over the graph, not the coordinate. The indexed readers’ cost tracks the records a window contains, growing 35.7× for cyvcf2 and 7.5× for bcftools over the same range. At 1 kb they win, cyvcf2 by 2.9× and bcftools by 2.2×; by 1 Mb QLever has overtaken cyvcf2, and at 10 Mb it leads both, by 10.6× and 2.9×. QLever’s index took 23.2s against 0.39s to bgzip and tabix-index the file, and the in-memory engines cost 896–1005 ms per window on the smaller graph against QLever’s 18.8 ms, so they were not carried further.

5 Discussion

5.1 When converting VCF to RDF pays, and when it does not

The use case answers the question the rest of the evaluation could not: what the graph lets a user do. It does not pay in lines: three of four realistic changes were data edits of the same size in a table-driven bcftools pipeline. Nor does it pay in speed: conversion and, for a whole genome, a 45-minute index come before the first question. It pays where the conventional route has no place to put a rule. Consents and cohort rules become statements beside the data that a query can ask about and a check can verify against the source VCF; a rule on a variant’s identity needs no code and holds whatever a file calls its contigs; and a frequency file joins the cohort and ClinVar through the same identifier, with no lookup written for it. A release view compared record for record with an oracle built from the VCF text is evidence, not a claim.

For questions about a single file the answer is conditional, and both conditions must be stated together. Once an index exists, SPARQL answers single and selective questions far faster than a parser’s scan, and its cost is flat in the width of a genomic window. But the parser wins any single cold question, one exhaustive pass, and narrow lookups against a tabix index. Counting the conversion of the 100,000-record slice

Table 10 What to build, by purpose, from the measurements in Section 4.

Purpose	Build
One pass of summaries per file	Nothing: a VCF tool is faster, and the graph never repays its conversion
Repeated or ad hoc questions, linking, governed release	Gzip N-Triples, served by QLever; its index costs about 1.3s per million triples
Archiving or shipping a graph	COTTAS, the smallest artifact ($0.37\text{--}0.55\times$ gzip N-Triples), or gzip N-Triples
Triple-pattern access to a file in place, without an index build	HDT or COTTAS, at fixture scale only

(423.7s, with HDT) against a saving of about 10.2s per question (11.5s for cyvcf2, a mean of 1.32s for QLever), the RDF route breaks even after about 44 questions; for a single batch it never does. For an analysis run once over one file, a VCF tool is the better choice, and for one run inside a cohort-scale variant store such as those in Section 1, that store is. Table 10 turns these measurements into advice.

5.2 What the representations buy

Under the default engine, HDT and COTTAS are transport and archive formats rather than query indexes. QLever decodes them and builds its own index, and the artifact changed its query time by 0.3% at 657M triples, while the engines that query the files in place were competitive only at fixture scale. The decision that sets query cost is the engine, not the representation. The representations’ cost, on the other hand, is real: for a whole genome they took 14.5 of the 16.07 conversion hours, and their outputs were 78 times the download. COTTAS earns its build time as the smallest artifact; HDT, larger than the gzip-framed N-Triples it was built from, earns it only where triple-pattern access to a file in place is the point.

The sample profiles trade queryability for size. The expanded profile makes every genotype a resource that a query, a shape or a policy can address, at 25 triples per sample per record, which a cohort cannot afford. The condensed profile is tractable but gives up that addressability: a genotype is a position in a vector literal, which a query must decode, a shape cannot constrain, and a policy cannot withhold without rewriting the vector. Because its values move into literals, its triple count also understates its size, so triple counts alone should not be used to compare RDF representations of genotype data.

5.3 What the validation establishes

The validation layer shows representation equivalence for the stated fields: on the tested inputs, a SPARQL engine and a VCF parser derive the same summaries, and the shapes hold. The division of labour between the two halves is the finding the mutation experiment makes precise. The queries compare what an oracle can compute independently; the shapes constrain what no oracle observes, such as which allele a genotype calls; and only together, with the full shape set, did they detect every

injected fault. That set self-joins the graph and runs only on inputs up to 16 MiB; on anything larger the default profile adds nothing the queries miss, which makes streaming implementations of those checks the first validation priority (Section 5.4). The layer also found real defects, which is the best evidence that it can fail. Its limits follow from the same design. Value-level comparisons ran on inputs of up to 17.1M triples, so the corpus files and the whole genome are supported by decode and triple-count checks only; detection was measured on 113 chosen mutations, not on every possible fault; and agreement with a VCF parser says nothing about whether the source calls are biologically correct.

5.4 Limitations and future work

The sample ladders hold INFO fixed, so they characterize structural scaling rather than cohort summaries, and the 2,504-sample call set was converted only in 10,000-record slices. The use case’s consents are simulated, and each participant is a single-sample file. No other converter was run on the same inputs. Four priorities follow. First, value-level validation should reach whole genomes and cohorts, which needs streaming or indexed implementations of the two costlier shape profiles. Second, governed release should reach multi-sample files, which needs sample-level masking in the condensed profile. Third, linking would gain from GA4GH VRS identifiers [43] beside SPDI, which do not depend on NCBI’s accessions, and from federation with existing SPARQL resources. Fourth, a measured comparison with other converters on a shared input, and a native VCF logical source for RML, would remove the two largest gaps in the comparison and the pipeline. A separate truth-concordance experiment on the GIAB inputs could test biological call accuracy, kept distinct from representation equivalence.

VCF-RDFizer turns VCF into a graph that can be checked against its source, linked to other files and resources through shared identifiers, and released under rules stated beside the data. That is worth its conversion cost when those properties are the point, and not otherwise.

Declarations

Conflicts of interest

None declared.

Funding

Elias Crum acknowledges support from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N).

[AUTHOR QUERY: Funding statements for B.B., R.T. and G.E.]

Data availability

The large VCF inputs are downloaded from their original public providers rather than redistributed in Git. Canonical filenames, source URLs/download commands, experiment manifests, query files, normalized outputs, and comparison reports are

maintained at <https://github.com/ecrum19/vcf-rdfizer-testing>. The archive provides run manifests and hashes for tracing the source inputs, generated artifacts, and executed queries. The use case of Section 4.2 uses only public inputs (the Personal Genome Project, Genome in a Bottle and precisionFDA genomes, the 1000 Genomes high-coverage call set, ClinVar and Ensembl 116), fetched by its harness with their digests pinned. Its case definitions, simulated consents, baseline, effort measurement and per-arm comparisons are in the testing repository’s experiment 17 (`benchmarks/use_case/acmg`).

Code availability

The VCF-RDFizer source is available at <https://github.com/ecrum19/VCF-RDFizer>. The target benchmark release is v3.1.0 (commit `d3b34d5`), distributed through PyPI, Conda-Forge, and Docker Hub; its image digest, pinned toolchain and the record of every benchmark cell are in Supplementary Section S1. Each run records its tool commit, command, and resolved image digest. The linkers and the policy plug-in (v0.2.0) evaluated in Section 4.2 are released in VCF-RDFizer v3.3.0

[AUTHOR QUERY: Add the v3.3.0 tag commit once the release is published.]

. Their tests beyond the tool’s own suite are kept separately, in the testing repository’s `plugin-tests` directory. Testing scripts, the integrated per-cell run record, normalized outputs, and paper artifacts are available at <https://github.com/ecrum19/vcf-rdfizer-testing>.

Author contributions statement

E.C. and G.E. conceived the tool and the experiments. E.C. implemented the conversion workflow and executed the experiments. E.C. and R.T. analyzed the results. E.C. wrote the manuscript draft. E.C., R.T., G.E., and B.B. reviewed and edited the manuscript.

Acknowledgments

All benchmarking computation was performed using the resources of IDLab at Ghent University.

Ethics approval and consent to participate

This study used only publicly released genomic data and involved no interaction with, or recruitment of, human participants. The five Personal Genome Project files were published by their donors under the project’s open-consent model [58]; the Genome in a Bottle and precisionFDA files are public reference materials [59, 62]; the 1000 Genomes Phase 3 and high-coverage call sets and HGSVC2 are released by their consortia for research reuse [60, 61, 64]; and ClinVar is a public NCBI resource [65]. The consents in Section 4.2 are simulated and attach to no real participant’s wishes. No attempt was made to identify any individual, and the results report only aggregate summaries and software-performance measurements. Ethics approval was therefore not sought.

Consent for publication

Not applicable. The manuscript contains no individual-level data, images or personal details; every reported figure is an aggregate count or a software-performance measurement over publicly released data.

Materials availability

No biological materials were generated or used. The digital materials needed to reproduce the study are public: VCF-RDFizer v3.1.0 (source on GitHub; packages on PyPI, Conda-Forge and Docker Hub, per-run commits and image digests in the benchmark archive); the VCF Core Vocabulary and its SHACL shapes [41]; the eleven difficult-input fixtures (`benchmarks/fixtures/awkward*.vcf`); the scripts that derive the record and sample ladders from the public corpus (`benchmarks/02_derive_ladders.sh`); and the download commands for all ten corpus files, all in <https://github.com/ecrum19/vcf-rdfizer-testing>. The corpus files themselves are not redistributed and are obtained from their original providers.

References

- [1] The Variant Call Format Specification (2024). <https://github.com/samtools/hts-specs>
- [2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). *Clinical Pharmacology & Therapeutics* **113**(5), 1036–1047 (2023) <https://doi.org/10.1002/cpt.2790>
- [3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeschirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nabbout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. *European Journal of Human Genetics* **30**(9), 1017–1021 (2022) <https://doi.org/10.1038/s41431-022-01113-x>
- [4] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation* **9**(1), 44 (2022) <https://doi.org/10.1038/s41439-022-00222-9>
- [5] Van Der Horst, E., Unni, D., Kopmels, F., Armida, J., Touré, V., Franke, W., Cramer, K., Cirillo, E., Osterle, S.: Bridging Clinical and Genomic Knowledge: An Extension of the SPHN RDF Schema for Seamless Integration and FAIRification of Omics Data. (2023). <https://doi.org/10.20944/preprints202312.0373>.

- [6] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. *GigaScience* **10**(2), 008 (2021) <https://doi.org/10.1093/gigascience/giab008>
- [7] Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1: Concepts and Abstract Syntax (2014). <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
- [8] Hogan, A., Blomqvist, E., Cochez, M., D’Amato, C., Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. *ACM Computing Surveys* **54**(4) (2021) <https://doi.org/10.1145/3447772>
- [9] World Wide Web Consortium (W3C): LinkedData. <https://www.w3.org/wiki/LinkedData> Accessed 2026-03-04
- [10] Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., Morissette, J.: Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics* **41**(5), 706–716 (2008) <https://doi.org/10.1016/j.jbi.2008.03.004>
- [11] Breit, A., Ott, S., Agibetov, A., Samwald, M.: OpenBioLink: A benchmarking framework for large-scale biomedical link prediction (2019) <https://doi.org/10.48550/ARXIV.1912.04616> . Publisher: arXiv Version Number: 2
- [12] SIB Swiss Institute of Bioinformatics RDF Group Members, Altenhoff, A., Bairoch, A., Bansal, P., Baratin, D., Bastian, F., Bolleman*, J., Bridge, A., Burdet, F., Crameri, K., Dauvillier, J., Dessimoz, C., Gehant, S., Glover, N., Gnodtke, K., Hayes, C., Ibberson, M., Kriventseva, E., Kuznetsov, D., Frédérique, L., Mehl, F., Mendes De Farias*, T., Michel, P.-A., Moretti, S., Morgat, A., Österle, S., Pagni, M., Redaschi, N., Robinson-Rechavi, M., Samarasinghe, K., Sima, A.-C., Szklarczyk, D., Topalov, O., Touré, V., Unni, D., Von Mering, C., Wollbrecht, J., Zahn-Zabal*, M., Zdobnov, E.: The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research* **52**(D1), 44–51 (2024) <https://doi.org/10.1093/nar/gkad902>
- [13] Harris, S., Seaborne, A., Prud’hommeaux, E.: SPARQL 1.1 Query Language (2013). <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- [14] Iannella, R., Villata, S.: ODRL Information Model 2.2. W3C (2018). <https://www.w3.org/TR/odrl-model/> Accessed 2025-11-27
- [15] Lindenbaum, P.: Jvarkit: java-based utilities for Bioinformatics (2015) <https://doi.org/10.6084/m9.figshare.1425030.v1>

- [16] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Version V4 (2020). <https://doi.org/10.5281/zenodo.3903111>
- [17] Penha, E.D.S., Iriabho, E., Dussaq, A., Oliveira, D.M., Almeida, J.S.: Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics* **33**(4), 547–548 (2017) <https://doi.org/10.1093/bioinformatics/btw652>
- [18] Katayama, T., Kawashima, S., Micklem, G., Kawano, S., Kim, J.-D., Kocbek, S., Okamoto, S., Wang, Y., Wu, H., Yamaguchi, A., *et al.*: Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research* **8**, 1677 (2019) <https://doi.org/10.12688/f1000research.18238.1>
- [19] Baran, J., Durgahee, B.S.B., Eilbeck, K., Antezana, E., Hoehndorf, R., Dumontier, M.: GFVO: the Genomic Feature and Variation Ontology. *PeerJ* **3**, 933 (2015) <https://doi.org/10.7717/peerj.933>
- [20] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology* **18**(5), 1009123 (2022) <https://doi.org/10.1371/journal.pcbi.1009123>
- [21] Bendiken, A., Bonnal, R.J.P., Strozzi, F.: ruby-rdf/rdf-vcf: RDF VCF parser (2025). <https://github.com/ruby-rdf/rdf-vcf> Accessed 2025-03-01
- [22] Prasanna, S., Kumar, A., Rao, D., Simoes, E.J., Rao, P.: A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data* **7**, 1466391 (2025) <https://doi.org/10.3389/fdata.2024.1466391>
- [23] Larmande, P., Pittolat, B., Tando, N., Pomie, Y., Happi Happi, B.G., Guignon, V., Ruiz, M.: AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data* **26**(Suppl 1), 73 (2025) <https://doi.org/10.1186/s12863-025-01359-6>
- [24] Bodrug-Schepers, A., Chabane, H., Montoya, G., Redon, R., Gaignard, A., Serrano-Alvarado, P.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. *CEUR Workshop Proceedings*, Barcelona, Spain (2025). <https://hal.science/hal-04908530v2/document> Accessed 2025-11-24
- [25] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03 (2019). <https://gitlab.com/roelj/sparqling-genomics>
- [26] Hail Team: Hail: scalable genomic data analysis. Accessed October 2026 (2026). <https://github.com/hail-is/hail>

- [27] Paila, U., Chapman, B.A., Kirchner, R., Quinlan, A.R.: GEMINI: integrative exploration of genetic variation and genome annotations. *PLoS Computational Biology* **9**(7), 1003153 (2013) <https://doi.org/10.1371/journal.pcbi.1003153>
- [28] TileDB, Inc.: TileDB-VCF: efficient variant-call data storage and retrieval. Accessed October 2026 (2026). <https://github.com/TileDB-Inc/TileDB-VCF>
- [29] Rambla, J., Baudis, M., Ariosa, R., *et al.*: Beacon v2 and Beacon networks: A “lingua franca” for federated data discovery in biomedical genomics, and beyond. *Human Mutation* **43**(6), 791–799 (2022) <https://doi.org/10.1002/humu.24369>
- [30] Kelleher, J., Lin, M., Albach, C.H., Birney, E., *et al.*: htsget: a protocol for securely streaming genomic data. *Bioinformatics* **35**(1), 119–121 (2019) <https://doi.org/10.1093/bioinformatics/bty492>
- [31] HL7 International: HL7 FHIR Genomics Reporting Implementation Guide. Accessed October 2026 (2026). <https://hl7.org/fhir/uv/genomics-reporting/>
- [32] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics* **1**(2), 100027 (2021) <https://doi.org/10.1016/j.xgen.2021.100027>
- [33] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics* **7**(1), 39 (2016) <https://doi.org/10.1186/s13326-016-0067-z>
- [34] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology* **6**(5), 44 (2005) <https://doi.org/10.1186/gb-2005-6-5-r44>
- [35] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). <https://rml.io/specs/rml/> Accessed 2025-11-24
- [36] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d’Amato, C. (eds.) *The Semantic Web – ISWC 2022* vol. 13489, pp. 697–713. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40 . Series Title: *Lecture Notes in Computer Science*. https://link.springer.com/10.1007/978-3-031-19433-7_40 Accessed 2025-11-24

- [37] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems* **37**(1), 1–30 (2019) <https://doi.org/10.1145/3231935>
- [38] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web* **19**, 22–41 (2013)
- [39] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: *The Semantic Web – ISWC 2025. Lecture Notes in Computer Science*, vol. 16141, pp. 313–331. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-09530-5_18 . https://doi.org/10.1007/978-3-032-09530-5_18
- [40] Crum, E., Buelens, B., Ertaylan, G., Taelman, R.: VCF Core Vocabulary: A Semantic Foundation for RDF Representations of VCF. Manuscript in preparation for SWAT4HCLS 2027 (2026)
- [41] Crum, E.: VCF Core Vocabulary. Version 2.1.3. GitHub repository. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026 (2026). <https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a>
- [42] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. *CEUR Workshop Proceedings*, Barcelona, Spain (2025). <https://www.dei.unipd.it/~silvello/papers/2025.MCS.SWAT4HCLS.pdf>
- [43] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., O'Neill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). <https://doi.org/10.5281/zenodo.15059674> . <https://doi.org/10.5281/zenodo.15059674> Accessed 2026-03-02
- [44] RDF/HDT contributors: hdt-cpp: HDT C++ Library and Tools. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest. <https://github.com/rdfhdt/hdt-cpp>
- [45] FRINK OKN: hdtc: HDT Creator. Computer software, MIT License. <https://github.com/frink-okn/hdtc>
- [46] Diefenbach, D., Giménez-García, J.M.: HDTCat: Let’s make HDT generation scale. In: *The Semantic Web – ISWC 2020. Lecture Notes in Computer Science*, vol. 12507, pp. 18–33. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-62466-8_2
- [47] Arenas-Guerrero, J.: pycottas: Python COTTAS library for compressing and

- querying RDF. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities (2026). <https://pypi.org/project/pycottas/> Accessed 2026-08-31
- [48] Crum, E.: Comunica SPARQL COTTAS: a Comunica Engine for Query Evaluation over COTTAS files. npm package `@elias.crum/query-sparql-cottas`. <https://github.com/ecrum19/comunica-feature-cottas>
 - [49] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. *Bioinformatics* **27**(15), 2156–2158 (2011) <https://doi.org/10.1093/bioinformatics/btr330>
 - [50] Pedersen, B.S., Quinlan, A.R.: *cyvcf2: fast, flexible variant analysis with Python*. *Bioinformatics* **33**(12), 1867–1869 (2017) <https://doi.org/10.1093/bioinformatics/btx057>
 - [51] Bast, H., Buchhold, B.: QLever: A query engine for efficient SPARQL+Text search. In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pp. 647–656 (2017). <https://doi.org/10.1145/3132847.3132921> . <https://doi.org/10.1145/3132847.3132921>
 - [52] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular SPARQL query engine for the web. In: *Proceedings of the 17th International Semantic Web Conference*, pp. 239–255. Springer, ??? (2018). https://doi.org/10.1007/978-3-030-00668-6_15 . <https://comunica.dev/>
 - [53] Comunica Association: Querying local files from the command line (2026). https://comunica.dev/docs/query/getting_started/query_cli_file/ Accessed 2026-08-31
 - [54] Comunica Association: HDT: Querying HDT files with Comunica (2026). <https://comunica.dev/docs/query/advanced/hdt/> Accessed 2026-08-31
 - [55] Holmes, J.B., Moyer, E., Phan, L., Maglott, D., Kattman, B.: SPDI: data model for variants and applications at NCBI. *Bioinformatics* **36**(6), 1902–1907 (2020) <https://doi.org/10.1093/bioinformatics/btz856>
 - [56] Xin, J., Mark, A., Afrasiabi, C., Tsueng, G., *et al.*: High-performance web services for querying gene and variant annotation. *Genome Biology* **17**, 91 (2016) <https://doi.org/10.1186/s13059-016-0953-9>
 - [57] Lawson, J., Cabili, M.N., Kerry, G., *et al.*: The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **1**(2), 100028 (2021) <https://doi.org/10.1016/j.xgen.2021.100028>
 - [58] Personal Genome Project (2005). https://my.pgp-hms.org/public_genetic_data

Accessed 2025-11-24

- [59] NIST Genome in a Bottle (2025). <https://www.nist.gov/programs-projects/genome-bottle> Accessed 2025-11-24
- [60] The 1000 Genomes Project Consortium: A global reference for human genetic variation. *Nature* **526**(7571), 68–74 (2015) <https://doi.org/10.1038/nature15393>
- [61] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. *Science* **372**(6537), 7117 (2021) <https://doi.org/10.1126/science.abf7117>
- [62] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small variants with linked and long reads. *Cell Genomics* **2**(5), 100128 (2022) <https://doi.org/10.1016/j.xgen.2022.100128>
- [63] Miller, D.T., Lee, K., Abul-Husn, N.S., Amendola, L.M., *et al.*: ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine* **25**(8), 100866 (2023) <https://doi.org/10.1016/j.gim.2023.100866>
- [64] Byrska-Bishop, M., Evani, U.S., Zhao, X., *et al.*: High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell* **185**(18), 3426–344019 (2022) <https://doi.org/10.1016/j.cell.2022.08.004>
- [65] National Center for Biotechnology Information: ClinVar VCF Release (GRCh38). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/